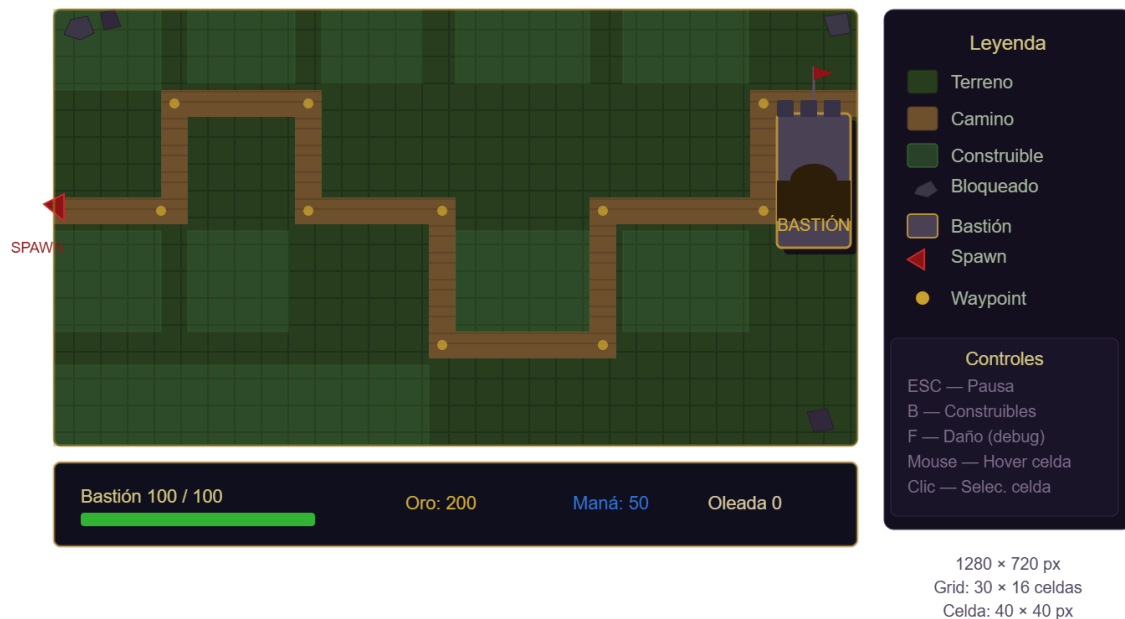


Primero leo el estado actual del proyecto para no contradecir nada existente. Tengo toda la información necesaria. El plan es claro: crear `MapSystem` con la geometría real del mapa, definir el camino con waypoints, las zonas construibles y el bastión como entidad propia; luego reemplazar el renderizado provisional de `gameplay.py` con el nuevo sistema. Empiezo. Ahora reescribo `gameplay.py` completo integrando los nuevos sistemas: 22/22 en verde. Ahora ejecuto la suite completa para verificar que nada se rompió: 59/59 tests pasando. Empaqueto y genero la visualización del escenario:---



Resumen del sprint

Archivos creados

Archivo	Tipo	Descripción
<code>data/maps/map_01.json</code>	Nuevo	Definición del mapa: waypoints del camino, celdas bloqueadas, posición del bastión
<code>src/systems/map_system.py</code>	Nuevo	Lógica del grid: tipos de celda, camino, zonas construibles, conversiones píxel↔celda
<code>src/systems/map_renderer.py</code>	Nuevo	Renderizado completo del escenario en capas: terreno, camino, bastión, rocas, spawn, hover
<code>tests/test_map_system.py</code>	Nuevo	22 pruebas unitarias — todas en verde

Archivos modificados

Archivo	Cambio
<code>src/core/settings.py</code>	Añadidas constantes <code>CELL_SIZE</code> , <code>GRID_COLS</code> , <code>GRID_ROWS</code> , <code>MAP_OFFSET_X/Y</code> , <code>HUD_HEIGHT</code> y clase <code>CellType</code>
<code>src/screens/gameplay.py</code>	Reemplazado el renderizado provisional por integración completa con <code>MapSystem</code> + <code>MapRenderer</code>

Tests totales: 59/59 en verde 

Explicación técnica de las decisiones clave

Separación MapSystem / MapRenderer. El `MapSystem` es lógica pura: no dibuja nada, solo responde preguntas (`is_buildable`, `get_cell_type`, `pixel_to_cell`). El `MapRenderer` solo dibuja: lee el estado del `MapSystem` y lo convierte en píxeles. Esta separación permite que el `BuildSystem` (siguiente sprint) consulte el mapa sin importar nada relacionado con Pygame, y los tests del `MapSystem` corren sin display activo.

Precálculo de surfaces (baking). El terreno base (30×16 = 480 celdas) y el overlay de zonas construibles se dibujan una sola vez en `pygame.Surface` fuera de pantalla al iniciar la partida. En cada frame solo se llama a `screen.blit()` una vez para cada capa, en lugar de iterar los 480 rectángulos. El bastión y el indicador de spawn no se pueden precalcular porque dependen de valores que cambian en tiempo real (HP del bastión, animación del pulso).

Camino definido en JSON con waypoints. Los 12 waypoints de `map_01.json` definen una ruta en S que atraviesa el mapa con cuatro cambios de dirección. El método `_expand_path()` interpola automáticamente todas las celdas entre cada par de waypoints consecutivos, tanto en segmentos horizontales como verticales. Si el diseñador quiere cambiar la ruta, solo modifica el JSON, sin tocar código.

CellType como fuente de verdad. Toda la lógica de comportamiento de celda se basa en el `CellType` almacenado en la matriz, no en coordenadas hardcodeadas. El `BuildSystem` del sprint siguiente solo necesita llamar a `is_buildable(col, row)` y `set_occupied(col, row)` para integrarse completamente.

Hover interactivo. `GameplayScreen` escucha `MOUSEMOTION`, convierte las coordenadas a celda con `pixel_to_cell()`, y pasa el resultado al renderer mediante `set_hover_cell()`. El renderer dibuja el overlay de color semitransparente: verde si la celda es construible, rojo si no lo es. Esta infraestructura ya está lista para que el `BuildSystem` use el mismo mecanismo para mostrar preview de torres.

Codigo de los archivos

data/maps/map_01.json

```
{
  "name": "Valle de Eldoria",
  "description": "El paso de montaña que conduce al Último Bastión.",
  "grid_cols": 30,
  "grid_rows": 16,

  "path_waypoints": [
    [0, 7],
    [4, 7],
    [4, 3],
    [9, 3],
    [9, 7],
    [14, 7],
    [14, 12],
    [20, 12],
    [20, 7],
    [26, 7],
    [26, 3],
    [29, 3]
  ],

  "blocked_cells": [
    [1, 0], [2, 0], [3, 0],
    [6, 0], [7, 0],
    [11, 0], [12, 0],
    [0, 15], [1, 15], [2, 15],
    [5, 15], [6, 15],
    [22, 0], [23, 0],
    [27, 0], [28, 0],
    [0, 1], [0, 2],
    [29, 14], [29, 15],
    [28, 15], [27, 15]
  ],

  "bastion_col": 28,
  "bastion_row": 1,
  "bastion_cols": 2,
  "bastion_rows": 5,

  "spawn_col": 0,
  "spawn_row": 7
}
```

"""

src/systems/map_system.py

=====

Sistema que gestiona el grid lógico del mapa de juego.

Responsabilidades

- Cargar la definición del mapa desde un archivo JSON.
- Mantener la matriz de celdas (CellType) que representa el estado del terreno en cada casilla.
- Calcular y exponer la lista de waypoints del camino enemigo como coordenadas en píxeles (centro de celda).
- Determinar si una celda es construible (GRASS libre).
- Marcar celdas como ocupadas cuando se coloca una defensa.
- No renderiza nada; el renderizado lo hace MapRenderer.

Integración

- GameplayScreen instancia MapSystem en on_enter().
- MapRenderer recibe una referencia al MapSystem para leer la matriz y dibujar cada frame.
- BuildSystem (sprint futuro) consultará is_buildable() antes de colocar torres.
- PathfindingSystem (sprint futuro) leerá get_path_waypoints() para calcular las rutas de los enemigos.

"""

```
from __future__ import annotations
```

```
import json
```

```
import logging
```

```
import os
```

```
from typing import NamedTuple
```

```
import pygame
```

```
from src.core.settings import (
    CELL_SIZE, GRID_COLS, GRID_ROWS,
    MAP_OFFSET_X, MAP_OFFSET_Y, CellType, Paths
)
```

```
logger = logging.getLogger(__name__)
```

```
# -----
```

```
# Tipos de datos de soporte
```

```
# -----
```

```
class Cell(NamedTuple):
```

```
    """Información de una celda individual del grid."""
```

```

cell_type: str      # Constante CellType
occupied: bool = False # True si hay una defensa colocada

# -----
# MapSystem
# -----

class MapSystem:
    """
    Gestiona el grid lógico del mapa y la geometría del escenario.

    Attributes:
        _grid:      Matriz [row][col] de objetos Cell.
        _path_waypoints: Lista de (col, row) que forman el camino.
        _bastion_rect: pygame.Rect que cubre las celdas del bastión.
        _spawn_cell: (col, row) de aparición de enemigos.
        map_name:    Nombre descriptivo del mapa cargado.
    """

    def __init__(self) -> None:
        self._grid: list[list[Cell]] = []
        self._path_waypoints: list[tuple[int, int]] = []
        self._path_cells: set[tuple[int, int]] = set()
        self._bastion_rect: pygame.Rect = pygame.Rect(0, 0, 0, 0)
        self._spawn_cell: tuple[int, int] = (0, 0)
        self.map_name: str = ""
        self._map_data: dict = {}

# -----
# Carga
# -----

def load(self, filename: str = "map_01.json") -> None:
    """
    Carga el mapa desde un archivo JSON en data/maps/.

    Args:
        filename: Nombre del archivo (sin ruta).
    """
    path = os.path.join(Paths.MAPS, filename)
    try:
        with open(path, "r", encoding="utf-8") as f:
            self._map_data = json.load(f)
            logger.info("Mapa '%s' cargado.", path)
    except FileNotFoundError:
        logger.error("Archivo de mapa no encontrado: %s — usando mapa por defecto.",
path)

```

```

        self._map_data = self._default_map_data()

    self._build_grid()

# -----
# Construcción del grid
# -----

def _build_grid(self) -> None:
    """Construye la matriz de celdas a partir de los datos del JSON."""
    data = self._map_data
    self.map_name = data.get("name", "Mapa sin nombre")

    cols = data.get("grid_cols", GRID_COLS)
    rows = data.get("grid_rows", GRID_ROWS)

    # 1 — Inicializar toda la cuadrícula como GRASS
    self._grid = [
        [Cell(CellType.GRASS) for _ in range(cols)]
        for _ in range(rows)
    ]

    # 2 — Trazar el camino (expandir waypoints a celdas individuales)
    waypoints = data.get("path_waypoints", [])
    self._path_waypoints = [(wp[0], wp[1]) for wp in waypoints]
    self._path_cells = self._expand_path(self._path_waypoints)

    for (col, row) in self._path_cells:
        if self._in_bounds(col, row):
            self._grid[row][col] = Cell(CellType.PATH)

    # 3 — Marcar celdas bloqueadas (decoración)
    for (col, row) in data.get("blocked_cells", []):
        if self._in_bounds(col, row):
            self._grid[row][col] = Cell(CellType.BLOCKED)

    # 4 — Marcar el área del bastión
    bc = data.get("bastion_col", GRID_COLS - 2)
    br = data.get("bastion_row", GRID_ROWS // 2 - 1)
    bcw = data.get("bastion_cols", 2)
    brh = data.get("bastion_rows", 3)

    for r in range(br, br + brh):
        for c in range(bc, bc + bcw):
            if self._in_bounds(c, r):
                self._grid[r][c] = Cell(CellType.BASTION)

    self._bastion_rect = pygame.Rect(

```

```

        MAP_OFFSET_X + bc * CELL_SIZE,
        MAP_OFFSET_Y + br * CELL_SIZE,
        bcw * CELL_SIZE,
        brh * CELL_SIZE,
    )

    # 5 — Guardar punto de spawn
    sc = data.get("spawn_col", 0)
    sr = data.get("spawn_row", GRID_ROWS // 2)
    self._spawn_cell = (sc, sr)

    logger.debug(
        "Grid construido: %dx%d — camino: %d celdas — bastión: %s",
        cols, rows, len(self._path_cells), self._bastion_rect
    )

# -----
# Geometría del camino
# -----

def _expand_path(
    self,
    waypoints: list[tuple[int, int]]
) -> set[tuple[int, int]]:
    """
    Convierte la lista de waypoints en el conjunto de todas las
    celdas que forman el camino, trazando líneas rectas entre
    cada par de waypoints consecutivos.

    Args:
        waypoints: Lista de (col, row) del JSON.

    Returns:
        Conjunto de (col, row) que pertenecen al camino.
    """
    cells: set[tuple[int, int]] = set()
    for i in range(len(waypoints) - 1):
        c0, r0 = waypoints[i]
        c1, r1 = waypoints[i + 1]
        # Segmento horizontal
        if r0 == r1:
            for c in range(min(c0, c1), max(c0, c1) + 1):
                cells.add((c, r0))
        # Segmento vertical
        elif c0 == c1:
            for r in range(min(r0, r1), max(r0, r1) + 1):
                cells.add((c0, r))
    return cells

```

```

# -----
# API pública — consultas
# -----

def get_cell_type(self, col: int, row: int) -> str:
    """Devuelve el CellType de la celda (col, row)."""
    if not self._in_bounds(col, row):
        return CellType.BLOCKED
    return self._grid[row][col].cell_type

def is_buildable(self, col: int, row: int) -> bool:
    """
    True si se puede colocar una defensa en (col, row).
    Una celda es construible si es GRASS y no está ocupada.
    """
    if not self._in_bounds(col, row):
        return False
    cell = self._grid[row][col]
    return cell.cell_type == CellType.GRASS and not cell.occupied

def set_occupied(self, col: int, row: int, occupied: bool = True) -> None:
    """Marca o libera una celda como ocupada por una defensa."""
    if self._in_bounds(col, row):
        original = self._grid[row][col]
        self._grid[row][col] = Cell(original.cell_type, occupied)

def get_path_waypoints_px(self) -> list[tuple[float, float]]:
    """
    Devuelve los waypoints del camino como coordenadas de píxel
    (centro de celda), listas para el pathfinding de enemigos.
    """
    return [self._cell_center_px(c, r) for c, r in self._path_waypoints]

def get_spawn_px(self) -> tuple[float, float]:
    """Posición en píxeles del punto de aparición de enemigos."""
    return self._cell_center_px(*self._spawn_cell)

def get_bastion_rect(self) -> pygame.Rect:
    """pygame.Rect del área del bastión en coordenadas de pantalla."""
    return self._bastion_rect.copy()

def get_bastion_center_px(self) -> tuple[float, float]:
    """Centro del bastión en píxeles."""
    r = self._bastion_rect
    return (r.x + r.width / 2, r.y + r.height / 2)

def pixel_to_cell(self, px: float, py: float) -> tuple[int, int]:

```



```

        """Convierte coordenadas de pantalla a (col, row) de celda."""
        col = int((px - MAP_OFFSET_X) / CELL_SIZE)
        row = int((py - MAP_OFFSET_Y) / CELL_SIZE)
        return col, row

def cell_to_pixel_topleft(self, col: int, row: int) -> tuple[int, int]:
    """Devuelve la esquina superior izquierda de la celda en píxeles."""
    return (
        MAP_OFFSET_X + col * CELL_SIZE,
        MAP_OFFSET_Y + row * CELL_SIZE,
    )

@property
def grid(self) -> list[list[Cell]]:
    """Acceso de solo lectura a la matriz de celdas."""
    return self._grid

@property
def path_waypoints(self) -> list[tuple[int, int]]:
    """Waypoints (col, row) del camino en coordenadas de celda."""
    return list(self._path_waypoints)

@property
def path_cells(self) -> set[tuple[int, int]]:
    """Conjunto de celdas que pertenecen al camino."""
    return set(self._path_cells)

@property
def spawn_cell(self) -> tuple[int, int]:
    """Celda de aparición de enemigos (col, row)."""
    return self._spawn_cell

# -----
# Internos
# -----

def _in_bounds(self, col: int, row: int) -> bool:
    rows = len(self._grid)
    cols = len(self._grid[0]) if rows else 0
    return 0 <= row < rows and 0 <= col < cols

def _cell_center_px(self, col: int, row: int) -> tuple[float, float]:
    return (
        MAP_OFFSET_X + col * CELL_SIZE + CELL_SIZE / 2,
        MAP_OFFSET_Y + row * CELL_SIZE + CELL_SIZE / 2,
    )

# -----

```

```
# Fallback si no existe el JSON
# -----
```

```
@staticmethod
def _default_map_data() -> dict:
    """Genera un mapa mínimo en memoria cuando no hay JSON disponible."""
    mid = GRID_ROWS // 2
    return {
        "name": "Mapa por defecto",
        "grid_cols": GRID_COLS,
        "grid_rows": GRID_ROWS,
        "path_waypoints": [
            [0, mid], [GRID_COLS - 3, mid]
        ],
        "blocked_cells": [],
        "bastion_col": GRID_COLS - 2,
        "bastion_row": mid - 1,
        "bastion_cols": 2,
        "bastion_rows": 3,
        "spawn_col": 0,
        "spawn_row": mid,
    }
```

```
"""
```

```
src/systems/map_renderer.py
```

```
=====
```

Responsable exclusivo del renderizado visual del escenario.

Separa el "qué hay en el mapa" (MapSystem) del "cómo se dibuja" (MapRenderer), siguiendo el principio de responsabilidad única.

Capas de renderizado (orden de pintado):

1. Terreno base — celdas GRASS con textura procedural
2. Camino — celdas PATH con efecto de tierra compacta
3. Zonas construibles — highlight sutil sobre GRASS libre
4. Celdas bloqueadas — rocas / decoración
5. Decoración — detalles visuales sobre el terreno
6. Bastión — estructura defensiva con estado de HP
7. Indicador de spawn — flecha en el punto de aparición enemiga
8. Overlay de hover — celda resaltada bajo el cursor (opcional)

Colores y estética: Fantasía Medieval Oscura / Pixel Art estilizado.

```
"""
```

```
from __future__ import annotations
import math
import logging
from typing import TYPE_CHECKING
```

```

import pygame

from src.core.settings import (
    CELL_SIZE, MAP_OFFSET_X, MAP_OFFSET_Y,
    CellType, Colors
)
from src.core.asset_loader import AssetLoader

if TYPE_CHECKING:
    from src.systems.map_system import MapSystem

logger = logging.getLogger(__name__)

# -----
# Paleta de colores del escenario
# -----

class MapColors:
    # Terreno
    GRASS_A    = (38, 58, 32) # Verde oscuro — casillas pares
    GRASS_B    = (32, 50, 26) # Verde más oscuro — casillas impares
    GRASS_DETAIL = (28, 44, 22) # Línea de celda sobre hierba

    # Camino
    PATH_CENTER = (110, 85, 48) # Tierra central
    PATH_EDGE   = (90, 68, 36)  # Borde del camino
    PATH_LINE   = (75, 55, 28)  # Línea divisoria del camino

    # Zonas construibles (overlay translúcido)
    BUILD_TINT   = (80, 160, 80, 30) # Verde muy sutil
    BUILD_BORDER = (60, 140, 60, 80) # Borde construible

    # Celdas bloqueadas
    ROCK_MAIN    = (55, 50, 60)
    ROCK_SHADOW  = (35, 30, 40)
    ROCK_LIGHT   = (80, 72, 88)

    # Bastión
    BASTION_WALL    = (75, 65, 90)
    BASTION_WALL_DMG = (105, 60, 30)
    BASTION_WALL_CRIT = (110, 25, 25)
    BASTION_MERLON   = (60, 52, 72)
    BASTION_GATE     = (45, 35, 20)
    BASTION_BORDER   = (180, 145, 55) # Dorado
    BASTION_FLAG     = (160, 20, 20)  # Rojo oscuro

```

```
# Spawn
SPAWN_ARROW = (160, 30, 30)
SPAWN_PULSE = (200, 50, 50, 120)
```

```
# Grid overlay
GRID_LINE = (0, 0, 0, 18)
```

```
class MapRenderer:
```

```
    """
```

```
    Dibuja el escenario completo a partir del estado del MapSystem.
```

```
    Attributes:
```

```
        _map: Referencia al MapSystem con la lógica del grid.
        _show_buildable: Si True, resalta las zonas construibles.
        _hover_cell: (col, row) de la celda bajo el cursor, o None.
        _tick: Acumulador de tiempo para animaciones.
        _build_surf: Surface precalculada de zonas construibles.
        _terrain_surf: Surface precalculada del terreno base.
```

```
    """
```

```
    def __init__(self, map_system: "MapSystem") -> None:
```

```
        self._map = map_system
        self._show_buildable: bool = True
        self._hover_cell: tuple[int, int] | None = None
        self._tick: float = 0.0
```

```
        # Surfaces precalculadas (se regeneran si cambia el mapa)
```

```
        self._terrain_surf: pygame.Surface | None = None
```

```
        self._build_surf: pygame.Surface | None = None
```

```
        self._bake_terrain()
```

```
        self._bake_buildable()
```

```
    # -----
```

```
    # Configuración pública
```

```
    # -----
```

```
    def set_hover_cell(self, col: int, row: int) -> None:
```

```
        """Actualiza la celda resaltada bajo el cursor del jugador."""
```

```
        self._hover_cell = (col, row)
```

```
    def clear_hover(self) -> None:
```

```
        """Elimina el resaltado de hover."""
```

```
        self._hover_cell = None
```

```
    def toggle_buildable_overlay(self) -> None:
```

```
        """Activa/desactiva la visualización de zonas construibles."""
```

```

self._show_buildable = not self._show_buildable

def invalidate(self) -> None:
    """Fuerza el recalcu de las surfaces precalculadas."""
    self._bake_terrain()
    self._bake_buildable()

# -----
# Update (animaciones)
# -----

def update(self, dt: float) -> None:
    """Avanza el acumulador de tiempo para animaciones del escenario."""
    self._tick += dt

# -----
# Punto de entrada de renderizado
# -----

def draw(
    self,
    screen: pygame.Surface,
    bastion_hp_pct: float = 1.0
) -> None:
    """
    Dibuja todas las capas del escenario.

    Args:
        screen: Surface principal de Pygame.
        bastion_hp_pct: Porcentaje de HP del bastión [0.0 – 1.0],
            usado para adaptar su color visual.
    """
    # 1 — Terreno base (surface precalculada)
    if self._terrain_surf:
        screen.blit(self._terrain_surf, (MAP_OFFSET_X, MAP_OFFSET_Y))

    # 2 — Overlay de zonas construibles (surface precalculada)
    if self._show_buildable and self._build_surf:
        screen.blit(self._build_surf, (MAP_OFFSET_X, MAP_OFFSET_Y))

    # 3 — Decoración de rocas sobre el terreno base
    self._draw_rocks(screen)

    # 4 — Bastión (necesita HP actual → no se puede precalcular)
    self._draw_bastion(screen, bastion_hp_pct)

    # 5 — Indicador animado del punto de spawn
    self._draw_spawn_indicator(screen)

```

```

# 6 — Resaltado del hover del cursor
if self._hover_cell:
    self._draw_hover(screen, *self._hover_cell)

# -----
# Precálculo (baking)
# -----

def _bake_terrain(self) -> None:
    """
    Dibuja el terreno base (GRASS + PATH + grid) en una surface
    fuera de pantalla que se reutiliza cada frame sin recalcular.
    """
    grid = self._map.grid
    if not grid:
        return

    rows = len(grid)
    cols = len(grid[0])
    w = cols * CELL_SIZE
    h = rows * CELL_SIZE

    surf = pygame.Surface((w, h), pygame.SRCALPHA)

    for row in range(rows):
        for col in range(cols):
            x = col * CELL_SIZE
            y = row * CELL_SIZE
            rect = pygame.Rect(x, y, CELL_SIZE, CELL_SIZE)
            ct = grid[row][col].cell_type

            if ct == CellType.PATH:
                self._draw_path_cell(surf, rect, col, row)
            elif ct == CellType.BASTION:
                # El área del bastión se pinta con la tierra
                # debajo del edificio
                pygame.draw.rect(surf, MapColors.GRASS_A, rect)
            else:
                # GRASS y BLOCKED comparten base de hierba
                color = MapColors.GRASS_A if (row + col) % 2 == 0 \
                    else MapColors.GRASS_B
                pygame.draw.rect(surf, color, rect)
                # Detalle de hierba: línea diagonal muy sutil
                if (row * 3 + col * 7) % 11 == 0:
                    pygame.draw.line(
                        surf, MapColors.GRASS_DETAIL,
                        (x + 6, y + CELL_SIZE - 8),

```

```

        (x + 14, y + CELL_SIZE - 14), 1
    )

# Grid overlay (líneas de celda)
grid_surf = pygame.Surface((w, h), pygame.SRCALPHA)
for row in range(rows + 1):
    pygame.draw.line(grid_surf, MapColors.GRID_LINE,
                     (0, row * CELL_SIZE), (w, row * CELL_SIZE), 1)
for col in range(cols + 1):
    pygame.draw.line(grid_surf, MapColors.GRID_LINE,
                     (col * CELL_SIZE, 0), (col * CELL_SIZE, h), 1)
surf.blit(grid_surf, (0, 0))

self._terrain_surf = surf
logger.debug("Terreno precalculado: %dx%d px", w, h)

def _bake_buildable(self) -> None:
    """
    Dibuja el overlay de zonas construibles en una surface
    semitransparente que se reutiliza cada frame.
    """
    grid = self._map.grid
    if not grid:
        return

    rows = len(grid)
    cols = len(grid[0])
    surf = pygame.Surface((cols * CELL_SIZE, rows * CELL_SIZE),
                          pygame.SRCALPHA)

    fill_color = MapColors.BUILD_TINT
    border_color = MapColors.BUILD_BORDER

    for row in range(rows):
        for col in range(cols):
            if self._map.is_buildable(col, row):
                x = col * CELL_SIZE
                y = row * CELL_SIZE
                rect = pygame.Rect(x, y, CELL_SIZE, CELL_SIZE)
                pygame.draw.rect(surf, fill_color, rect)
                pygame.draw.rect(surf, border_color, rect, 1)

    self._build_surf = surf

# -----
# Dibujo por capa
# -----

```

```

def _draw_path_cell(
    self,
    surf: pygame.Surface,
    rect: pygame.Rect,
    col: int,
    row: int
) -> None:
    """Dibuja una celda de camino con tierra y borde desgastado."""
    # Relleno tierra
    pygame.draw.rect(surf, MapColors.PATH_CENTER, rect)

    # Borde oscuro (sensación de bordillo)
    pygame.draw.rect(surf, MapColors.PATH_EDGE, rect, 2)

    # Marcas de pisadas (líneas sutiles longitudinales)
    cx = rect.x
    cy = rect.y
    cs = CELL_SIZE
    # Detectar si el segmento es horizontal o vertical
    path_cells = self._map.path_cells
    is_h = (col - 1, row) in path_cells or (col + 1, row) in path_cells
    is_v = (col, row - 1) in path_cells or (col, row + 1) in path_cells

    if is_h and not is_v:
        pygame.draw.line(surf, MapColors.PATH_LINE,
                         (cx + 4, cy + cs // 3),
                         (cx + cs - 4, cy + cs // 3), 1)
        pygame.draw.line(surf, MapColors.PATH_LINE,
                         (cx + 4, cy + 2 * cs // 3),
                         (cx + cs - 4, cy + 2 * cs // 3), 1)
    elif is_v and not is_h:
        pygame.draw.line(surf, MapColors.PATH_LINE,
                         (cx + cs // 3, cy + 4),
                         (cx + cs // 3, cy + cs - 4), 1)
        pygame.draw.line(surf, MapColors.PATH_LINE,
                         (cx + 2 * cs // 3, cy + 4),
                         (cx + 2 * cs // 3, cy + cs - 4), 1)
    else:
        # Esquina / cruce — cruz simple
        mid_x = cx + cs // 2
        mid_y = cy + cs // 2
        pygame.draw.circle(surf, MapColors.PATH_LINE, (mid_x, mid_y), 3)

def _draw_rocks(self, screen: pygame.Surface) -> None:
    """Dibuja decoración de rocas sobre celdas BLOCKED."""
    grid = self._map.grid
    for row, row_cells in enumerate(grid):
        for col, cell in enumerate(row_cells):

```



```

        if cell.cell_type != CellType.BLOCKED:
            continue
        x = MAP_OFFSET_X + col * CELL_SIZE
        y = MAP_OFFSET_Y + row * CELL_SIZE
        self._draw_rock(screen, x, y, (col * 7 + row * 13) % 4)

def _draw_rock(
    self,
    screen: pygame.Surface,
    x: int, y: int,
    variant: int
) -> None:
    """
    Dibuja una roca de forma procedural.
    variant (0–3) varía la forma para dar diversidad visual.
    """
    cs = CELL_SIZE
    cx = x + cs // 2
    cy = y + cs // 2

    offsets = [
        [(0, -10), (10, -4), (8, 6), (-2, 10), (-10, 4), (-8, -6)],
        [(0, -12), (8, -6), (12, 2), (6, 10), (-4, 12), (-10, 4), (-8, -6)],
        [(-6, -8), (6, -10), (12, 0), (8, 8), (-2, 10), (-10, 4)],
        [(0, -8), (8, -8), (10, 4), (4, 10), (-6, 8), (-10, 0), (-6, -6)],
    ]
    pts_shadow = [(cx + dx + 2, cy + dy + 2) for dx, dy in offsets[variant]]
    pts_main = [(cx + dx, cy + dy) for dx, dy in offsets[variant]]
    pts_hi = [(cx + dx - 2, cy + dy - 2) for dx, dy in offsets[variant][:3]]

    pygame.draw.polygon(screen, MapColors.ROCK_SHADOW, pts_shadow)
    pygame.draw.polygon(screen, MapColors.ROCK_MAIN, pts_main)
    pygame.draw.polygon(screen, MapColors.ROCK_LIGHT, pts_hi)
    pygame.draw.polygon(screen, MapColors.ROCK_SHADOW, pts_main, 1)

def _draw_bastion(
    self,
    screen: pygame.Surface,
    hp_pct: float
) -> None:
    """
    Dibuja el bastión con nivel de daño visual basado en HP.

    Tres estados:
    · Sano (> 60 %): color de piedra normal
    · Dañado (30–60 %): tono anaranjado-quemado
    · Crítico (< 30 %): rojo intenso con grietas
    """

```

```
rect = self._map.get_bastion_rect()

# Color del cuerpo según HP
if hp_pct > 0.60:
    wall_color = MapColors.BASTION_WALL
elif hp_pct > 0.30:
    wall_color = MapColors.BASTION_WALL_DMG
else:
    wall_color = MapColors.BASTION_WALL_CRIT
```

```
cs = CELL_SIZE
bx, by = rect.x, rect.y
bw, bh = rect.width, rect.height
```

```
# — Sombra de la estructura
```

```
shadow_rect = pygame.Rect(bx + 4, by + 4, bw, bh)
pygame.draw.rect(screen, (20, 15, 25), shadow_rect, border_radius=4)
```

```
# — Cuerpo principal
```

```
pygame.draw.rect(screen, wall_color, rect, border_radius=4)
```

```
# — Almenas (merlones) — franja superior
```

```
merlon_h = cs // 4
merlon_w = cs // 2
for mx in range(bx, bx + bw, merlon_w + 4):
    merlon_rect = pygame.Rect(mx, by - merlon_h, merlon_w, merlon_h)
    pygame.draw.rect(screen, MapColors.BASTION_MERLON,
                     merlon_rect, border_radius=2)
```

```
# — Puerta (arco en la fila del camino)
```

```
# Buscar qué celda del bastión está sobre el camino
gate_y = by + bh // 2 - cs // 2
gate_rect = pygame.Rect(bx, gate_y, bw, cs)
pygame.draw.rect(screen, MapColors.BASTION_GATE, gate_rect)
# Arco de la puerta
arc_rect = pygame.Rect(bx + bw // 4, gate_y, bw // 2, cs)
pygame.draw.ellipse(screen, MapColors.BASTION_GATE, arc_rect)
```

```
# — Borde dorado
```

```
pygame.draw.rect(screen, MapColors.BASTION_BORDER, rect,
                 width=2, border_radius=4)
```

```
# — Bandera en la cima
```

```
flag_x = bx + bw // 2
```

```

flag_y = by - merlon_h - cs // 2
pygame.draw.line(screen, MapColors.ROCK_MAIN,
                 (flag_x, flag_y), (flag_x, by - merlon_h), 2)
pygame.draw.polygon(screen, MapColors.BASTION_FLAG, [
    (flag_x,    flag_y),
    (flag_x + 14, flag_y + 5),
    (flag_x,    flag_y + 10),
])

```

— Grietas en estado crítico

```

if hp_pct < 0.30:
    crack_color = (30, 20, 25)
    pygame.draw.line(screen, crack_color,
                    (bx + bw // 3, by + 8),
                    (bx + bw // 3 + 6, by + bh // 3), 1)
    pygame.draw.line(screen, crack_color,
                    (bx + 2 * bw // 3, by + bh // 3),
                    (bx + 2 * bw // 3 - 8, by + 2 * bh // 3), 1)

```

— Etiqueta y HP

```

font_lbl = AssetLoader.get_font(None, 12)
lbl = font_lbl.render("BASTIÓN", True, Colors.GOLD)
screen.blit(lbl, lbl.get_rect(center=(bx + bw // 2, by + bh // 2 + 10)))

```

```

def _draw_spawn_indicator(self, screen: pygame.Surface) -> None:
    """

```

Dibuja una flecha animada (pulsante) en el punto de spawn de los enemigos, en el borde izquierdo del camino.

```

    """
    sc, sr = self._map.spawn_cell
    sx = MAP_OFFSET_X + sc * CELL_SIZE
    sy = MAP_OFFSET_Y + sr * CELL_SIZE + CELL_SIZE // 2

```

Pulso: alpha oscila entre 120 y 220 con el tiempo

```

alpha = int(170 + 50 * math.sin(self._tick * 3.0))

```

```

arrow_surf = pygame.Surface((24, 30), pygame.SRCALPHA)
pts = [(0, 15), (16, 4), (16, 26)]
pygame.draw.polygon(arrow_surf, (*MapColors.SPAWN_ARROW, alpha), pts)
pygame.draw.polygon(arrow_surf, (200, 60, 60, alpha), pts, 1)
screen.blit(arrow_surf, (sx - 2, sy - 15))

```

```

def _draw_hover(
    self,
    screen: pygame.Surface,
    col: int,

```

```

        row: int
    ) -> None:
        """Resalta la celda bajo el cursor con un borde semitransparente."""
        x = MAP_OFFSET_X + col * CELL_SIZE
        y = MAP_OFFSET_Y + row * CELL_SIZE
        rect = pygame.Rect(x, y, CELL_SIZE, CELL_SIZE)

        is_build = self._map.is_buildable(col, row)
        color = (80, 200, 80, 100) if is_build else (200, 80, 80, 80)

        hl = pygame.Surface((CELL_SIZE, CELL_SIZE), pygame.SRCALPHA)
        hl.fill(color)
        screen.blit(hl, (x, y))

        border_color = (100, 220, 100) if is_build else (220, 80, 80)
        pygame.draw.rect(screen, border_color, rect, 2)

"""
tests/test_map_system.py
=====
Pruebas unitarias del MapSystem.
No requieren Pygame ni display activo.
"""

import sys
import unittest
from unittest.mock import MagicMock

# Mockear pygame antes de cualquier import del proyecto
pygame_mock = MagicMock()
pygame_mock.Rect = lambda x, y, w, h: MagicMock(x=x, y=y, width=w, height=h)
sys.modules["pygame"] = pygame_mock

from src.systems.map_system import MapSystem
from src.core.settings import CellType, GRID_COLS, GRID_ROWS

def _minimal_map_data(waypoints=None, bastion_col=28, bastion_row=6,
                      blocked=None) -> dict:
    return {
        "name": "Test Map",
        "grid_cols": GRID_COLS,
        "grid_rows": GRID_ROWS,
        "path_waypoints": waypoints or [[0, 7], [5, 7], [5, 3], [10, 3]],
        "blocked_cells": blocked or [],
        "bastion_col": bastion_col,
        "bastion_row": bastion_row,
        "bastion_cols": 2,

```

```

    "bastion_rows": 3,
    "spawn_col": 0,
    "spawn_row": 7,
}

```

```

class TestMapSystemGrid(unittest.TestCase):

```

```

    def setUp(self):
        self.ms = MapSystem()
        self.ms._map_data = _minimal_map_data()
        self.ms._build_grid()

```

```

    def test_grid_dimensions(self):
        self.assertEqual(len(self.ms.grid), GRID_ROWS)
        self.assertEqual(len(self.ms.grid[0]), GRID_COLS)

```

```

    def test_default_cell_is_grass(self):
        # Una celda que no es camino ni bastión ni bloqueada
        ct = self.ms.get_cell_type(15, 0)
        self.assertEqual(ct, CellType.GRASS)

```

```

    def test_path_cells_are_path_type(self):
        # (0,7) a (5,7) → fila 7, columnas 0–5 son PATH
        for col in range(6):
            with self.subTest(col=col):
                self.assertEqual(self.ms.get_cell_type(col, 7), CellType.PATH)

```

```

    def test_vertical_path_segment(self):
        # (5,7) a (5,3) → columna 5, filas 3–7 son PATH
        for row in range(3, 8):
            with self.subTest(row=row):
                self.assertEqual(self.ms.get_cell_type(5, row), CellType.PATH)

```

```

    def test_bastion_cells_are_bastion_type(self):
        # bastion_col=28, bastion_row=6, 2x3 celdas
        for r in range(6, 9):
            for c in range(28, 30):
                with self.subTest(r=r, c=c):
                    self.assertEqual(self.ms.get_cell_type(c, r), CellType.BASTION)

```

```

    def test_blocked_cells_are_blocked(self):
        ms2 = MapSystem()
        ms2._map_data = _minimal_map_data(blocked=[[2, 0], [3, 0]])
        ms2._build_grid()
        self.assertEqual(ms2.get_cell_type(2, 0), CellType.BLOCKED)
        self.assertEqual(ms2.get_cell_type(3, 0), CellType.BLOCKED)

```

```

def test_out_of_bounds_returns_blocked(self):
    self.assertEqual(self.ms.get_cell_type(-1, 0), CellType.BLOCKED)
    self.assertEqual(self.ms.get_cell_type(999, 0), CellType.BLOCKED)
    self.assertEqual(self.ms.get_cell_type(0, 999), CellType.BLOCKED)

```

```

class TestMapSystemBuildable(unittest.TestCase):

```

```

    def setUp(self):
        self.ms = MapSystem()
        self.ms._map_data = _minimal_map_data()
        self.ms._build_grid()

```

```

    def test_grass_is_buildable(self):
        # (15, 0) debe ser GRASS libre
        self.assertTrue(self.ms.is_buildable(15, 0))

```

```

    def test_path_is_not_buildable(self):
        self.assertFalse(self.ms.is_buildable(0, 7))

```

```

    def test_bastion_is_not_buildable(self):
        self.assertFalse(self.ms.is_buildable(28, 6))

```

```

    def test_blocked_is_not_buildable(self):
        ms2 = MapSystem()
        ms2._map_data = _minimal_map_data(blocked=[[1, 1]])
        ms2._build_grid()
        self.assertFalse(ms2.is_buildable(1, 1))

```

```

    def test_set_occupied_marks_not_buildable(self):
        self.ms.set_occupied(15, 0, True)
        self.assertFalse(self.ms.is_buildable(15, 0))

```

```

    def test_clear_occupied_restores_buildable(self):
        self.ms.set_occupied(15, 0, True)
        self.ms.set_occupied(15, 0, False)
        self.assertTrue(self.ms.is_buildable(15, 0))

```

```

    def test_set_occupied_out_of_bounds_does_not_crash(self):
        try:
            self.ms.set_occupied(999, 999, True)
        except Exception as e:
            self.fail(f"set_occupied fuera de límites lanzó excepción: {e}")

```

```

class TestMapSystemPathWaypoints(unittest.TestCase):

```

```

    def setUp(self):

```

```

self.ms = MapSystem()
self.ms._map_data = _minimal_map_data(
    waypoints=[[0, 8], [5, 8], [5, 3], [10, 3]]
)
self.ms._build_grid()

def test_waypoints_count(self):
    # 4 waypoints en el JSON
    self.assertEqual(len(self.ms.path_waypoints), 4)

def test_first_waypoint(self):
    self.assertEqual(self.ms.path_waypoints[0], (0, 8))

def test_last_waypoint(self):
    self.assertEqual(self.ms.path_waypoints[-1], (10, 3))

def test_spawn_cell(self):
    self.assertEqual(self.ms.spawn_cell, (0, 7))

def test_path_cells_are_non_empty(self):
    self.assertGreater(len(self.ms.path_cells), 0)

```

```

class TestMapSystemPixelConversions(unittest.TestCase):

```

```

    def setUp(self):
        self.ms = MapSystem()
        self.ms._map_data = _minimal_map_data()
        self.ms._build_grid()

    def test_pixel_to_cell_origin(self):
        from src.core.settings import MAP_OFFSET_X, MAP_OFFSET_Y, CELL_SIZE
        col, row = self.ms.pixel_to_cell(
            MAP_OFFSET_X + CELL_SIZE * 2 + CELL_SIZE // 2,
            MAP_OFFSET_Y + CELL_SIZE * 3 + CELL_SIZE // 2
        )
        self.assertEqual(col, 2)
        self.assertEqual(row, 3)

    def test_cell_to_pixel_topleft(self):
        from src.core.settings import MAP_OFFSET_X, MAP_OFFSET_Y, CELL_SIZE
        px, py = self.ms.cell_to_pixel_topleft(4, 2)
        self.assertEqual(px, MAP_OFFSET_X + 4 * CELL_SIZE)
        self.assertEqual(py, MAP_OFFSET_Y + 2 * CELL_SIZE)

```

```

class TestMapSystemDefaultFallback(unittest.TestCase):

```

```

def test_default_map_loads_without_json(self):
    """load() con archivo inexistente usa mapa por defecto sin lanzar excepción."""
    ms = MapSystem()
    try:
        ms.load("archivo_que_no_existe.json")
    except Exception as e:
        self.fail(f"load() con archivo inexistente lanzó: {e}")

    self.assertGreater(len(ms.grid), 0)
    self.assertGreater(len(ms.path_cells), 0)

if __name__ == "__main__":
    unittest.main()

"""
src/core/settings.py
=====
Constantes globales y configuración de runtime del videojuego.
Centraliza todos los valores de configuración para facilitar
ajustes sin modificar otros módulos.
"""

# -----
# Ventana y pantalla
# -----
SCREEN_WIDTH: int = 1280
SCREEN_HEIGHT: int = 720
SCREEN_TITLE: str = "El Último Bastión"
TARGET_FPS: int = 60

# -----
# Colores (R, G, B)
# -----
class Colors:
    BLACK    = (0, 0, 0)
    WHITE    = (255, 255, 255)
    DARK_GRAY = (30, 30, 30)
    GRAY     = (80, 80, 80)
    LIGHT_GRAY = (160, 160, 160)

# Paleta temática Fantasía Medieval Oscura
DARK_STONE = (40, 35, 45) # Fondo principal
DEEP_PURPLE = (55, 25, 75) # Acento oscuro
BLOOD_RED = (140, 20, 20) # Peligro / enemigos
GOLD = (212, 175, 55) # Recurso oro
MANA_BLUE = (50, 120, 220) # Recurso maná
FOREST_GREEN = (34, 85, 34) # Elementos naturales

```



```

    PARCHMENT = (220, 200, 160) # Texto principal UI

# -----
# Paths de assets (relativos a la raíz del proyecto)
# -----
class Paths:
    ASSETS = "assets"
    SPRITES = "assets/sprites"
    SOUNDS = "assets/sounds"
    FONTS = "assets/fonts"
    DATA = "data"
    MAPS = "data/maps"

# -----
# Parámetros del Game Loop
# -----
MAX_DELTA_TIME: float = 0.05 # Tope de dt en segundos (evita saltos por pausas del
SO)

# -----
# Identificadores de estados (pantallas)
# -----
class States:
    MAIN_MENU = "main_menu"
    GAMEPLAY = "gameplay"
    PAUSE = "pause"
    DEFEAT = "defeat"

# -----
# Eventos globales del bus
# -----
class Events:
    QUIT_GAME = "quit_game" # Cierra la aplicación
    RESTART_GAME = "restart_game" # Reinicia la partida desde derrota
    BASTION_DEFEAT = "bastion_defeat" # El bastión llegó a 0 HP

# -----
# Grid del mapa de juego
# -----
CELL_SIZE: int = 40 # píxeles por celda
GRID_COLS: int = 30 # columnas jugables
GRID_ROWS: int = 16 # filas jugables (16 * 40 = 640 px)
MAP_OFFSET_X: int = 0 # esquina X del grid
MAP_OFFSET_Y: int = 0 # esquina Y del grid
HUD_HEIGHT: int = 80 # banda inferior del HUD

# -----
# Tipos de celda del mapa

```

```
# -----
class CellType:
    GRASS    = "grass"    # terreno libre → construible
    PATH     = "path"     # camino de enemigos → NO construible
    BASTION  = "bastion"   # área del bastión → reservada
    BLOCKED  = "blocked"   # decoración / roca → NO construible
```

```
"""
```

```
src/screens/gameplay.py
```

```
=====
```

Pantalla principal de juego.

Coordina el renderizado del escenario (MapSystem + MapRenderer), mantiene el estado de la partida y gestiona los controles del jugador. Los sistemas de oleadas, enemigos y torres se conectarán en los siguientes sprints como dependencias inyectadas sin necesidad de modificar esta clase.

Controles:

ESC → Pantalla de Pausa

F → Aplica 25 de daño al bastión (debug)

B → Activa/desactiva overlay de zonas construibles

Mouse → Hover sobre celdas del mapa (muestra tipo de celda)

Integración con EventBus:

Publica: Events.BASTION_DEFEAT cuando bastión HP = 0

Escucha: Events.RESTART_GAME para reiniciar la partida

```
"""
```

```
from __future__ import annotations
import logging
```

```
import pygame
```

```
from src.screens.base_screen import BaseScreen
```

```
from src.core.settings import (
    SCREEN_WIDTH, SCREEN_HEIGHT,
    Colors, States, Events,
    MAP_OFFSET_X, MAP_OFFSET_Y,
    CELL_SIZE, HUD_HEIGHT,
)
```

```
from src.core.asset_loader import AssetLoader
```

```
from src.systems.map_system import MapSystem
```

```
from src.systems.map_renderer import MapRenderer
```

```
logger = logging.getLogger(__name__)
```

```
class GameplayScreen(BaseScreen):
```

```
    """
```

Pantalla de juego activo.

Attributes:

```
    _map:      Sistema lógico del mapa (grid + camino).
    _renderer: Renderizador del escenario.
    _bastion_hp: Puntos de vida actuales del bastión.
    _max_hp:    Puntos de vida máximos del bastión.
    _gold:      Cantidad de oro disponible (placeholder).
    _mana:      Cantidad de maná disponible (placeholder).
    _wave_num:  Número de oleada actual (placeholder).
```

```
    """
```

```
# -----
# Ciclo de vida
# -----
```

```
def on_enter(self) -> None:
```

```
    """
```

Inicializa (o reinicia) el estado completo de la partida:

1. Carga el mapa desde JSON.
2. Crea el renderer del escenario.
3. Resetea contadores de HP, recursos y oleada.
4. Suscribe eventos del bus.

```
    """
```

```
# — Mapa y renderer
```

```
self._map      = MapSystem()
self._map.load("map_01.json")
self._renderer = MapRenderer(self._map)
```

```
# — Estado de la partida
```

```
self._bastion_hp: int    = 100
self._max_hp: int        = 100
self._gold: int          = 200
self._mana: int          = 50
self._wave_num: int      = 0
self._defeat_triggered: bool = False
```

```
# — Hover del cursor
```

```
self._hovered_cell: tuple[int, int] | None = None
self._hovered_type: str = ""
```

— Suscribir reinicio

```
self.event_bus.subscribe(Events.RESTART_GAME, self._on_restart)
logger.info("GameplayScreen: partida iniciada — mapa '%s'.",
            self._map.map_name)
```

```
def on_exit(self) -> None:
    """Desuscribir eventos al salir para evitar duplicados."""
    self.event_bus.unsubscribe(Events.RESTART_GAME, self._on_restart)
```

```
# -----
# Game loop
# -----
```

```
def handle_events(self, events: list[pygame.event.Event]) -> None:
    for event in events:
        if event.type == pygame.KEYDOWN:
            self._handle_keydown(event.key)

        elif event.type == pygame.MOUSEMOTION:
            self._update_hover(event.pos)

        elif event.type == pygame.MOUSEBUTTONDOWN:
            if event.button == 1:
                self._on_left_click(event.pos)
```

```
def update(self, dt: float) -> None:
    """
    Actualiza la lógica del juego y las animaciones del renderer.
    Los sistemas de oleadas / combate se conectarán aquí.
    """
    self._renderer.update(dt)

    if self._bastion_hp <= 0 and not self._defeat_triggered:
        self._trigger_defeat()
```

```
def draw(self, screen: pygame.Surface) -> None:
    # Escenario completo
    self._renderer.draw(screen, self._bastion_hp / self._max_hp)
    # HUD encima del mapa
    self._draw_hud(screen)
    # Pistas de control
    self._draw_controls_hint(screen)
    # Info de celda bajo el cursor
    if self._hovered_cell:
        self._draw_cell_tooltip(screen)
```

```
# -----
```

```

# Manejo de inputs
# -----

def _handle_keydown(self, key: int) -> None:
    if key == pygame.K_ESCAPE:
        self.state_machine.change(States.PAUSE)

    elif key == pygame.K_f:
        # Debug: daño directo al bastión
        self._apply_damage(25)
        logger.debug("DEBUG daño bastión → HP=%d", self._bastion_hp)

    elif key == pygame.K_b:
        # Alternar overlay de zonas construibles
        self._renderer.toggle_buildable_overlay()

def _update_hover(self, mouse_pos: tuple[int, int]) -> None:
    """Actualiza la celda resaltada según la posición del cursor."""
    col, row = self._map.pixel_to_cell(*mouse_pos)
    grid = self._map.grid
    if 0 <= row < len(grid) and 0 <= col < len(grid[0]):
        self._hovered_cell = (col, row)
        self._hovered_type = self._map.get_cell_type(col, row)
        self._renderer.set_hover_cell(col, row)
    else:
        self._hovered_cell = None
        self._hovered_type = ""
        self._renderer.clear_hover()

def _on_left_click(self, mouse_pos: tuple[int, int]) -> None:
    """
    Placeholder del sistema de construcción.
    En el sprint de torres este método delegará al BuildSystem.
    """
    col, row = self._map.pixel_to_cell(*mouse_pos)
    if self._map.is_buildable(col, row):
        logger.debug("Clic en celda construible (%d, %d).", col, row)
    else:
        cell_type = self._map.get_cell_type(col, row)
        logger.debug("Clic en celda NO construible (%d, %d) — tipo: %s.",
                    col, row, cell_type)

# -----
# Lógica del bastión
# -----

def _apply_damage(self, amount: int) -> None:
    """Reduce la vida del bastión (mínimo 0)."""

```

```
self._bastion_hp = max(0, self._bastion_hp - amount)
```

```
def _trigger_defeat(self) -> None:
```

```
    """Publica el evento de derrota y transiciona a Defeat."""
```

```
    self._defeat_triggered = True
```

```
    logger.info("Bastión destruido — %s publicado.", Events.BASTION_DEFEAT)
```

```
    self.event_bus.publish(Events.BASTION_DEFEAT)
```

```
    self.state_machine.change(States.DEFEAT)
```

```
def _on_restart(self, _data=None) -> None:
```

```
    """Reinicia toda la partida cuando llega Events.RESTART_GAME."""
```

```
    logger.info("Reiniciando partida.")
```

```
    self.on_enter()
```

```
# -----
```

```
# HUD
```

```
# -----
```

```
def _draw_hud(self, screen: pygame.Surface) -> None:
```

```
    """
```

```
    Franja inferior con:
```

```
    · Barra de vida del bastión con porcentaje
```

```
    · Oro y Maná (placeholder)
```

```
    · Número de oleada
```

```
    """
```

```
    hud_y = SCREEN_HEIGHT - HUD_HEIGHT
```

```
    hud_w = SCREEN_WIDTH
```

```
# Fondo semitransparente
```

```
    hud_sf = pygame.Surface((hud_w, HUD_HEIGHT), pygame.SRCALPHA)
```

```
    hud_sf.fill((18, 12, 28, 220))
```

```
    screen.blit(hud_sf, (0, hud_y))
```

```
# Línea separadora dorada
```

```
    pygame.draw.line(screen, Colors.GOLD,
```

```
                    (0, hud_y), (hud_w, hud_y), 2)
```

```
    y_mid = hud_y + HUD_HEIGHT // 2
```

```
    font = AssetLoader.get_font(None, 20)
```

```
# — Barra de HP
```

```
self._draw_hp_bar(screen, 16, hud_y + 14, 230, 20)
```

```
# — Separador
```

```
pygame.draw.line(screen, (70, 60, 90),
```

```
                (262, hud_y + 10), (262, hud_y + HUD_HEIGHT - 10), 1)
```

— Oro

gold_s = font.render(f'Oro: {self._gold}', True, Colors.GOLD)
screen.blit(gold_s, gold_s.get_rect(midleft=(280, y_mid)))

— Maná

mana_s = font.render(f'Maná: {self._mana}', True, Colors.MANA_BLUE)
screen.blit(mana_s, mana_s.get_rect(midleft=(430, y_mid)))

— Oleada

wave_s = font.render(f'Oleada {self._wave_num}', True, Colors.PARCHMENT)
screen.blit(wave_s, wave_s.get_rect(midright=(hud_w - 16, y_mid)))

```
def _draw_hp_bar(
    self,
    screen: pygame.Surface,
    x: int, y: int, width: int, height: int
) -> None:
    """Barra de HP del bastión con color dinámico."""
    pct = self._bastion_hp / self._max_hp

    if pct > 0.60:
        bar_color = (50, 185, 50)
    elif pct > 0.30:
        bar_color = (205, 165, 30)
    else:
        bar_color = Colors.BLOOD_RED

    # Fondo
    pygame.draw.rect(screen, (35, 30, 45),
                     pygame.Rect(x, y, width, height), border_radius=3)
    # Relleno
    fill_w = max(0, int(width * pct))
    if fill_w > 0:
        pygame.draw.rect(screen, bar_color,
                         pygame.Rect(x, y, fill_w, height),
                         border_radius=3)
    # Borde
    pygame.draw.rect(screen, Colors.LIGHT_GRAY,
                     pygame.Rect(x, y, width, height), 1, border_radius=3)

    # Etiqueta encima de la barra
    lbl_font = AssetLoader.get_font(None, 14)
```

```

lbl = lbl_font.render(
    f"Bastión {self._bastion_hp} / {self._max_hp}",
    True, Colors.PARCHMENT
)
screen.blit(lbl, (x, y - 17))

def _draw_controls_hint(self, screen: pygame.Surface) -> None:
    """Guía de controles en la esquina superior derecha."""
    font = AssetLoader.get_font(None, 14)
    hints = [
        "ESC — Pausa",
        "B — Zonas construibles",
        "F — Daño (debug)",
    ]
    for i, text in enumerate(hints):
        s = font.render(text, True, Colors.GRAY)
        screen.blit(s, s.get_rect(topright=(SCREEN_WIDTH - 8, 8 + i * 17)))

def _draw_cell_tooltip(self, screen: pygame.Surface) -> None:
    """Muestra el tipo de celda bajo el cursor en la esquina inferior izquierda del HUD."""
    if not self._hovered_cell:
        return
    col, row = self._hovered_cell
    label_map = {
        "grass": ("Terreno libre", Colors.FOREST_GREEN),
        "path": ("Camino enemigo", Colors.BLOOD_RED),
        "bastion": ("Bastión", Colors.GOLD),
        "blocked": ("Bloqueado", Colors.LIGHT_GRAY),
    }
    text, color = label_map.get(
        self._hovered_type,
        (self._hovered_type, Colors.PARCHMENT)
    )
    font = AssetLoader.get_font(None, 14)
    info = f"({col},{row}) {text}"
    buildable = "[construible]" if self._map.is_buildable(col, row) else ""
    s = font.render(info + buildable, True, color)
    screen.blit(s, (8, SCREEN_HEIGHT - HUD_HEIGHT - 20))

```